

Ejercicio 1

Consumer que muestra un String

```
import java.util.function.Consumer;

public class Main {

    public static void main(String[] args) {

        Consumer<String> mostrar =

            texto -> System.out.println(texto);

        mostrar.accept("Estoy en clase de programación");

    }

}
```

Ejercicio 2

Mostrar lista de enteros con Consumer

```
import java.util.Arrays;
import java.util.List;
import java.util.function.Consumer;

public class Main {

    public static void main(String[] args) {

        List<Integer> numeros = Arrays.asList(

            1,2,3,4,5,6,7,8,9,10

        );

        Consumer<Integer> mostrar =

            numero -> System.out.println(numero);

        numeros.forEach(mostrar);

    }

}
```

Ejercicio 3

Mostrar última letra de cada String

```
import java.util.Arrays;
import java.util.List;
import java.util.function.Consumer;

public class Main {

    public static void main(String[] args) {

        List<String> palabras = Arrays.asList(

            "Java",

            "Programacion",

            "Lambda",

            "Clase",

            "Ordenador",

            "Monitor",

            "Teclado",

            "Raton",

            "Alumno",

            "Profesor"

        );

        Consumer<String> ultimaLetra =

            texto -> System.out.println(

                texto.charAt(texto.length() - 1)

            );

        palabras.forEach(ultimaLetra);

    }

}
```

Ejercicio 4

Supplier con String

```
import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) {

        Supplier<String> texto =

            () -> "Java es un lenguaje de programación";

        System.out.println(texto.get());

    }

}
```

Ejercicio 5

Supplier con número random

```
import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) {

        Supplier<Double> random =

            () -> Math.random();

        System.out.println(random.get());

    }

}
```

Ejercicio 6

Supplier de HashMap

```
import java.util.HashMap;

import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) {

        Supplier<HashMap<String, String>> crearMapa =

            () -> new HashMap<>();

    }

}
```

```

HashMap<String, String> mapa =
    crearMapa.get();
mapa.put("Nombre", "Adrian");
mapa.put("Curso", "Programacion");
System.out.println(mapa);
}
}

```

Ejercicio 7

Lambda multiplicación

```

import java.util.function.BiConsumer;

public class Main {

    public static void main(String[] args) {

        BiConsumer<Integer, Integer> multiplicar =

            (a, b) ->

                System.out.println(a * b);

        multiplicar.accept(-9, 7);

    }

}

```

Ejercicio 8

Lambdas suma, resta y división

```

import java.util.function.BiConsumer;

public class Main {

    public static void main(String[] args) {

        BiConsumer<Integer, Integer> suma =

            (a, b) ->

                System.out.println(a + b);

        BiConsumer<Integer, Integer> resta =

            (a, b) ->

                System.out.println(a - b);

    }

}

```

```

        BiConsumer<Integer, Integer> division =
            (a, b) ->
                System.out.println(a / b);
        suma.accept(-9, 7);
        resta.accept(-9, 7);
        division.accept(-9, 7);
    }
}

```

Ejercicio 9

Método calculator

```

import java.util.function.BiFunction;

public class Main {

    public static int calculator(
        int a,
        int b,
        BiFunction<Integer, Integer, Integer> operacion) {
        return operacion.apply(a, b);
    }

    public static void main(String[] args) {
        int suma = calculator(-9, 7,
            (a, b) -> a + b);
        int resta = calculator(-9, 7,
            (a, b) -> a - b);
        int multiplicacion = calculator(-9, 7,
            (a, b) -> a * b);
        System.out.println(suma);
        System.out.println(resta);
        System.out.println(multiplicacion);
    }
}

```

```
}
```

Ejercicio 10

LinkedHashMap con lambda

```
import java.util.LinkedHashMap;

public class Main {

    public static void main(String[] args) {

        LinkedHashMap<Integer, String> mapa =

            new LinkedHashMap<>();

        mapa.put(1, "Java");

        mapa.put(2, "Python");

        mapa.put(3, "C++");

        mapa.put(4, "JavaScript");

        mapa.put(5, "Kotlin");

        mapa.forEach(

            (clave, valor) ->

                System.out.println(

                    clave + " -> " + valor

                )

        );

    }

}
```

Ejercicio 11

Supplier con fecha actual

```
import java.time.LocalDate;

import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) {

        Supplier<LocalDate> fechaActual =

            () -> LocalDate.now();

    }

}
```

```
        System.out.println(fechaActual.get());
    }
}
```

Ejercicio 12

Clase Programador

```
import java.time.LocalDate;

public class Programador {

    private String nombre;
    private double salario;
    private LocalDate fechaInicio;

    public Programador(
        String nombre,
        double salario,
        LocalDate fechaInicio) {
        this.nombre = nombre;
        this.salario = salario;
        this.fechaInicio = fechaInicio;
    }

    public Programador(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public String toString() {
        return nombre + " - "
            + salario + " - "
            + fechaInicio;
    }
}
```

Crear objeto completo

```

import java.time.LocalDate;
import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) {

        Supplier<Programador> programador =

            () -> new Programador(

                "Adrian",

                50000,

                LocalDate.now()

            );

        System.out.println(programador.get());

    }

}

```

Ejercicio 13

Crear Programador solo con nombre

```

import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) {

        Supplier<Programador> programador =

            () -> new Programador("Adrian");

        System.out.println(programador.get());

    }

}

```

Ejercicio 14

Método fabricaProgramadores

```

import java.time.LocalDate;
import java.util.function.Supplier;

public class Main {

    public static Programador fabricaProgramadores(

```



```

        Supplier<Programador> proveedor) {
    Programador p = proveedor.get();
    if (p.getSalario() == 0) {
        p.setSalario(50000);
    }
    if (p.getFechaInicio() == null) {
        p.setFechaInicio(LocalDate.now());
    }
    return p;
}

public static void main(String[] args) {
    Programador p1 =
        fabricaProgramadores(
            () -> new Programador(
                "Adrian",
                60000,
                LocalDate.now()
            )
        );
    Programador p2 =
        fabricaProgramadores(
            () -> new Programador("Mario")
        );
    System.out.println(p1);
    System.out.println(p2);
}
}

```

Añadir getters y setters en Programador

```

public double getSalario() {

```

```
        return salario;
    }

    public void setSalario(double salario) {
        this.salario = salario;
    }

    public LocalDate getFechaInicio() {
        return fechaInicio;
    }

    public void setFechaInicio(LocalDate fechaInicio) {
        this.fechaInicio = fechaInicio;
    }
}
```